

Engineering Progress Report

EcoBuck Smart Compost Monitoring System — Assignment C (Connectivity & Backend)

Prepared by: Toqeer Ahmed

Role: Backend & AI Engineer

Date: July 8, 2026

Status: Architecture Design Phase Completed (Awaiting Implementation Approval)

1. Cover Page

- **Project Title:** EcoBuck Smart Compost Monitoring System
 - **Component:** Assignment C – Connectivity, Database & Backend API Services
 - **Target Repository:** <https://github.com/toqeer-ahmed/ecobuck-backend-connectivity>
 - **Version:** 1.0.0-Design Draft
-

2. Project Overview

EcoBuck is an IoT-enabled smart bin designed to optimize organic waste composting through real-time telemetry monitoring. By tracking temperature, humidity, and composting states directly from hardware, the system guides users through composting phases, issues alert notifications when environmental conditions drift out of bounds, and awards users "EcoBucks" upon successful compost batch completion.

3. Assignment Objectives

- Configure the backend infrastructure to ingest telemetry payloads from physical or simulated ESP32 device modules.
- Validate incoming measurements and establish time-series log tables inside a scalable database.
- Implement a Compost Rules Engine that runs safety parameter evaluations to detect biological anomalies.

- Expose secure REST APIs supporting registration, claims, configs, history analytics, and validation resets for mobile applications.
-

4. Scope of Responsibility

My role centers on the design, implementation, testing, and deployment of the backend connectivity platform. The scope includes: * Ingestion API design for incoming hardware telemetry events. * NoSQL database design matching Firestore query constraints. * Alerts evaluation and push notification integrations. * Mobile client application dashboard support APIs. * Local simulation client development for end-to-end integration checks.

5. Work Completed

- **Documentation Study:** Thoroughly studied the Assignment C specifications.
 - **Firmware Analysis:** Reviewed the Firmware team's implementation structures, FSM state boundaries, and JSON telemetry payload formatting.
 - **Integration Boundary Defined:** Identified the entry point where hardware HTTP POST requests reach the backend and established the validation boundary.
 - **Technology Analysis:** Evaluated frameworks and selected FastAPI for asynchronous logic and schema safety.
 - **Architecture & Design:** Completed the high-level system architecture, folder layouts, Firestore document structures, and REST API specification.
 - **Planning:** Drafted the 6-week daily sprint roadmap and personal learning roadmap.
 - **Workspace Setup:** Created the GitHub repository, configured gitignore exclusions, and initialized the directories layout.
-

6. System Architecture Designed

The architecture follows clean design principles, decoupling inputs from database entities:

- **ESP32 Firmware / Simulator Client:** Dispatches structured JSON telemetry payloads via HTTP POST parameters.
- **FastAPI Ingest API Controller:** Intercepts payload and triggers token verification check filters.

- **Pydantic Schema Validation:** Evaluates formatting rules and passes variables to repositories.
 - **Cloud Firestore Database:** Logs time-series data to subcollections and caches dashboard parameters in latest-status paths.
 - **Compost Alert Engine:** Runs safety thresholds check limits asynchronously.
 - **FCM Notification Gateway:** Dispatches simulated pushes to mobile clients if environmental conditions violate bounds.
-

7. Backend Module Breakdown

The project structure uses a clean layer separation layout: * `/app/api/v1/endpoints/`: Routing controllers separating telemetry ingestion from mobile dashboard views. * `/app/core/`: Application settings, environment configuration loaders, and custom global error filters. * `/app/schemas/`: Pydantic validation contracts ensuring strict parsing of email, authentication, telemetry, and claim models. * `/app/db/`: Connection initializers for Firebase Admin SDK. * `/app/repositories/`: Query isolation classes separating database interactions from API route logic. * `/app/alert_engine/`: Composting biological safety checks, rules parameters, and trigger handlers. * `/app/services/`: Simulated notification gateways representing FCM. * `/tests/`: Automated test suite for endpoints. * `/simulator/`: Self-contained composting curve simulation script.

8. Database Design Summary

We design Google Cloud Firestore NoSQL collections using a write-duplication caching strategy to keep reads low-cost: * `/users/{userId}`: Profiles containing display name, email, claimed devices array, and FCM tokens. * `/devices/{deviceId}`: Hardware profiles mapping owning user, label, FSM state, and dynamic configs. * `/devices/{deviceId}/telemetry/{telemetryId}`: Subcollection containing raw/filtered time-series logs. * `/devices/{deviceId}/latest/status`: Cached document written alongside telemetry posts, letting the mobile client read current conditions with a single read cost. * `/devices/{deviceId}/alerts/{alertId}`: Logged alerts and resolution timestamps.

9. API Design Summary

Our OpenAPI (Swagger) interface exposes: * `POST /api/v1/telemetry`: Firmware ingestion endpoint (expects `x-Device-Token` header, returns 202 Accepted). * `GET /api/v1/devices`: Lists user-claimed bins. * `POST /api/v1/devices/claim`: Pairs MAC hardware IDs to user profiles. * `GET /api/v1/devices/{id}/latest`: Loads cached dashboard values. * `GET /api/v1/devices/{id}/history`:

Queries time-series data for history graphs. * `POST /api/v1/devices/{id}/validate`: Triggers manual compost maturity checks. * `PUT /api/v1/devices/{id}/config`: Modifies calibration offsets and sleep timers.

10. Collaboration with Firmware Team

- **Data Exchange:** Backend expects MAC ID as `device_id`, alongside timestamp, temperature metrics (raw, filtered, trend average), humidity metrics, status tags, and cycle age.
 - **Security Header:** Handled via custom request header token `X-Device-Token`.
 - **Agreement Owner:** Backend and Firmware team leads will co-sign the schema validation parameters before final production deployment.
-

11. Collaboration Plan with Mobile Team

- **Dashboard Loading:** Mobile client listens to the `/latest/status` document cache path for low-cost, real-time telemetry updates.
 - **Warning Triggers:** Mobile client reads `/alerts` subcollection to display active system warning cards.
 - **Agreement Owner:** Backend and Mobile team leads will freeze response schema validation objects to avoid blocking app layouts.
-

12. GitHub Repository

- **Repository Name:** [Repository Name Placeholder]
 - **Repository URL:** [Repository URL Placeholder]
-

13. Technologies Selected

- **Programming Language:** Python 3.10+
- **Web Framework:** FastAPI (Uvicorn HTTP server)
- **Data Parsing:** Pydantic V2 (Pydantic Settings config)

- **Database:** Google Cloud Firestore (Firebase Admin SDK)
 - **Unit Testing:** Pytest / HTTPX TestClient
-

14. Current Learning Progress

- Completed initial study on FastAPI dependency injection lifecycles.
 - Learned NoSQL schema optimization rules, indexing mechanisms, and subcollection layouts.
 - Explored standard token validation patterns.
-

15. Challenges Encountered

- **NoSQL Optimization:** Designing Firestore schemas to avoid expensive linear queries when fetching time-series logs for charts. Resolved by defining indexed composite keys and a latest-status caching document.
 - **Device Authentication Design:** Creating a security checking system for the device that prevents unauthorized writes without imposing heavy cryptographic parsing loads on the low-power ESP32 microcontroller. Resolved by implementing header token lookups.
-

16. Next Development Phase

Upon approval of the folder structure, API specification contracts, and Firestore database layouts, the development phase will start. This phase will set up the production database connection, deploy local test suites, and connect simulated ESP32 clients.

17. Upcoming Sprint Plan

- **Week 1:** Repository initialization, core configuration loading, and custom exception filter middleware mapping.
- **Week 2:** Data validation schemas (Pydantic) implementation and Firestore Admin SDK connection setups.

- **Week 3:** Ingestion routes development and compost rules engine integration.
 - **Week 4:** Dashboard REST endpoints implementation (claims, configs, history analytics).
 - **Week 5:** Integration testing using mock hardware simulator suites.
 - **Week 6:** Unit tests execution, documentation preparation, and deployment scripts setup.
-

18. Learning Outcomes

- Understood clean repository patterns for NoSQL databases.
 - Gained experience decoupling Web layer endpoints from Core business logic.
 - Learned how to coordinate contracts across Firmware, Mobile, and Backend engineering boundaries.
-

19. Conclusion

The architecture design phase for EcoBuck's Connectivity & Backend (Assignment C) is completed. The proposed architecture decouples the database, rules engine, and API layers to ensure scalability and ease of future AI/ML model integrations. I am awaiting review and approval to proceed with the core development phase.

Team Members

- **Backend & AI Lead:** [Team Member Placeholder]
- **Firmware Lead:** [Team Member Placeholder]
- **Mobile App Lead:** [Team Member Placeholder]